Cervical Cancer Prediction System

Machine Learning Project Presentation

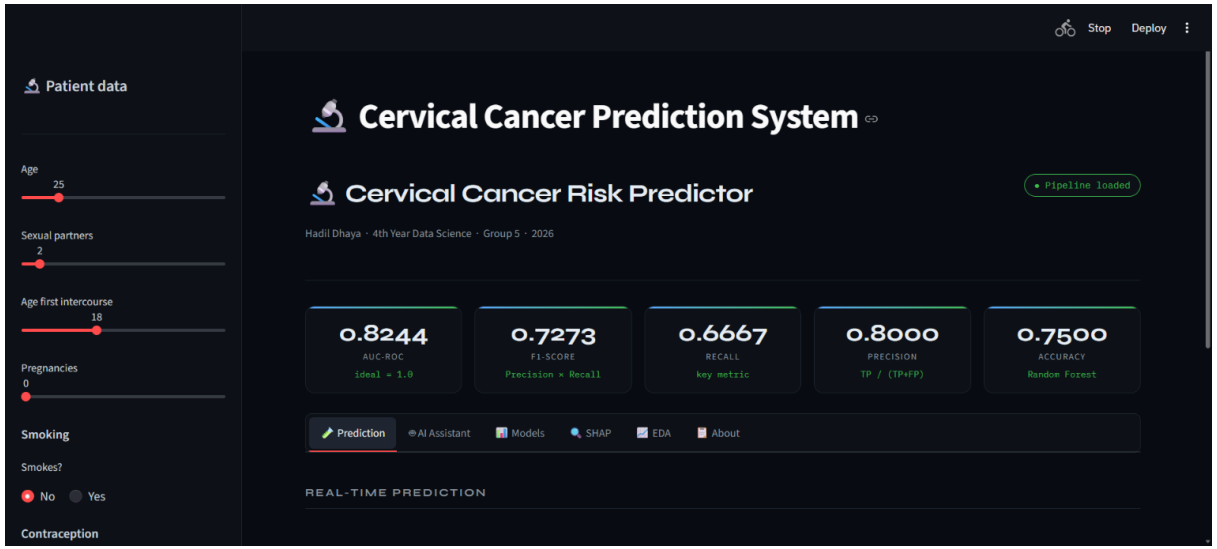
Author: Hadil Dhaya

From raw clinical data to a cleaned, deployed Random Forest pipeline with explainable predictions and UI validation.

Target variable: Biopsy | Deployed on Streamlit

Real project captures are used across the deck: interface, prediction screens, model comparison, SHAP, EDA and chatbot.

New first capture



Problem and Objective

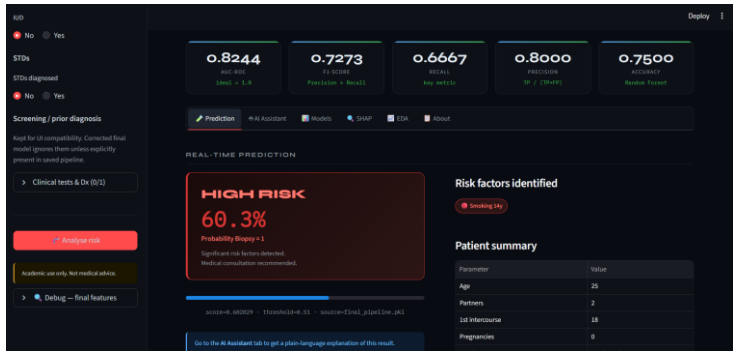
Why this project matters and what the system is designed to do

PROBLEM

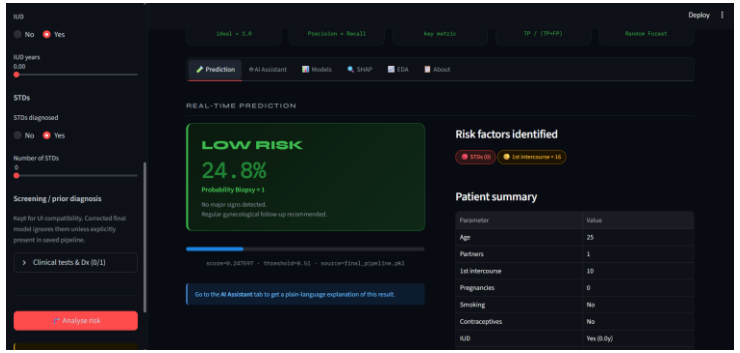
Support cervical cancer screening by helping analyze the target variable: Biopsy.
Biopsy.
Turn clinical risk-factor data into a usable prediction that is simple to read for beginners.
Provide a deployable interface so users can test patient profiles and review model outputs.
Reduce misleading results caused by weak data, leakage, missing values, noisy features and early illogical predictions.
Add a chatbot assistant to explain results and improve usability.

WHO BENEFITS?

Students building end-to-end ML systems
Users who need a simple prediction interface
Anyone learning why trustworthy ML is more than a single metric



Prediction capture: high-risk output from the deployed interface



Prediction capture: low-risk output used for sanity checks

Dataset and Data Challenges

From the original UCI cervical cancer dataset to a cleaned working file

Original source

UCI Cervical Cancer Risk Factors
858 rows · 36 columns · target = Biopsy

KEY ISSUES FOUND IN THE RAW DATA

Missing values and inconsistent formatting across multiple clinical variables. variables.
Duplicate rows and weak / low-impact features.
Leakage risk from near-diagnostic columns such as Hinselmann, Schiller, Citology and Dx fields.
Class imbalance in the original dataset, then a cleaned working file used for controlled experimentation.
Early outputs sometimes looked unrealistic, which signaled data or pipeline problems that needed review.

WORKING FILE USED IN THE FINAL WORKFLOW

Mydata.csv: 300 rows, 10 columns, 9 input features + Biopsy.
Balanced working dataset: 150 positive / 150 negative.
Used to simplify experimentation and make prediction behavior easier to inspect.

Dataset sample (working file)

Age	Partners	1stSex	Preg	Smoke	Years	Packs	Horm	HYears	Biopsy
16.0	1	15	0	0	0.0	0.0	1	0.42	1
38.5	3	18	2	1	10.0	4.0	1	11.18	1
40.0	1	17	4	0	0.0	0.0	1	15.00	1
17.0	1	15	0	0	0.0	0.0	1	0.25	0

Real working dataset used in the app and notebooks: Mydata.csv

EDA capture: class balance and age distribution from the interface

Technologies, Models, Methodology, and Deployment

How the system was built, evaluated and made usable online

TECHNOLOGIES

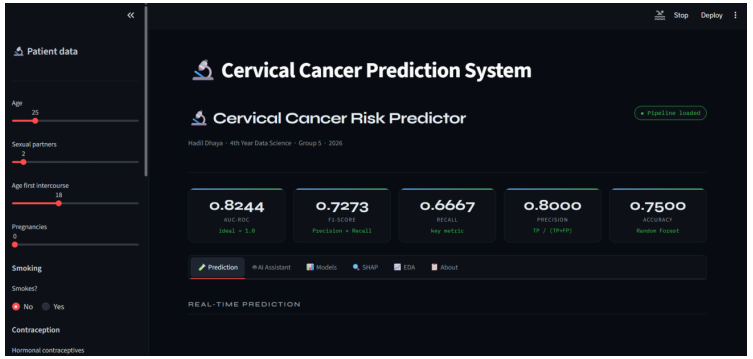
Python, Jupyter Notebook, Pandas, NumPy
Matplotlib, Seaborn, SHAP
Scikit-learn Pipelines, ColumnTransformer, StandardScaler, SimpleImputer
Train/validation/test split, cross-validation, RandomizedSearchCV
FastAPI backend + Streamlit frontend
Git / GitHub + Render deployment

MODELS TESTED

Logistic Regression — simple baseline, but weaker separation
KNN — useful local baseline, but sensitive to scaling / neighborhood choices
SVM (RBF) — powerful margin learner, but less stable here
Gradient Boosting / AdaBoost — strong nonlinear baselines
MLP Neural Network — flexible, but harder to stabilize on this dataset
Random Forest — best overall balance and more realistic behavior after cleaning

METHODOLOGY WORKFLOW

1. Collect & inspect data
2. Clean values, remove duplicates, leakage and weak features
3. Split train / validation / test correctly
4. Train several models and compare
5. Select Random Forest, tune threshold, deploy on Render



Deployed interface capture (Render-hosted app)

WHY RANDOM FOREST?

Most reliable and balanced performance among the tested models
Handled noise and feature interactions better than linear baselines
More stable after data cleaning and pipeline correction
Produced stronger evaluation results and more believable prediction behavior

Ī ÑCEÞÕPCEVĪÕŇ Ì ŎŇÑÕĪ ÑØŅŎØÖMĪÕŃÑ

Evaluation, prediction checks and why the final pipeline is more trustworthy

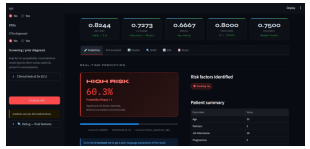
0.8244	0.7273	0.6667	ĆBĐĆĆĆ	0.7500
AUC-ROC	F1-score	Recall	Precision	Accuracy

MODEL COMPARISON ON THE TEST INTERFACE

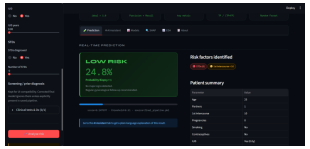
Confusion matrices and model comparison: Random Forest achieved the strongest balance

PREDICTION REVIEW & FIXES

Early experiments revealed suspicious or unstable outputs, so predictions were reviewed critically instead of trusting metrics alone.
Main causes investigated: leakage, weak features, bad preprocessing, class balance effects and threshold choice.
After pipeline correction, sanity checks compared low-risk and higher-risk profiles directly in the deployed app.



Example: higher-risk profile



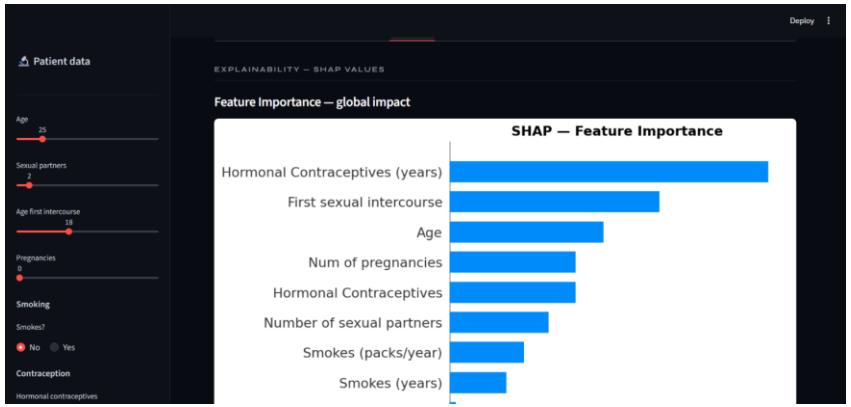
Example: lower-risk profile

Challenges Solved and Lessons Learned

What changed technically, and what the project taught

PROBLEMS SOLVED

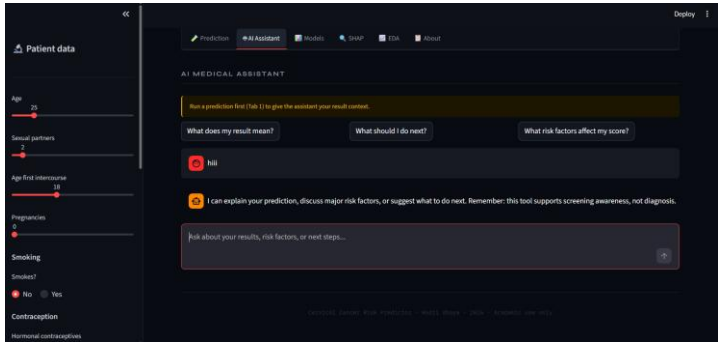
Detected and removed leakage-prone diagnostic columns.
Improved weak data handling with imputation inside the pipeline.
Reviewed class balance effects and avoided trusting accuracy alone.
Compared several models before choosing the final one.
Tested prediction logic in the UI instead of relying only on notebook outputs.
Made the system usable online with a deployed frontend and API flow.



SHAP capture: global feature importance for the final model

WHAT I LEARNED

Good ML is not just high metrics — predictions must also be logical, realistic and usable.
Feature importance and SHAP help explain why a model behaves the way it does.
The right model is chosen by trustworthiness and balance, not by one number only.
Deployment exposes issues that notebooks alone can hide.



Chatbot capture: user-facing explanation support inside the application

Conclusion and Future Improvements

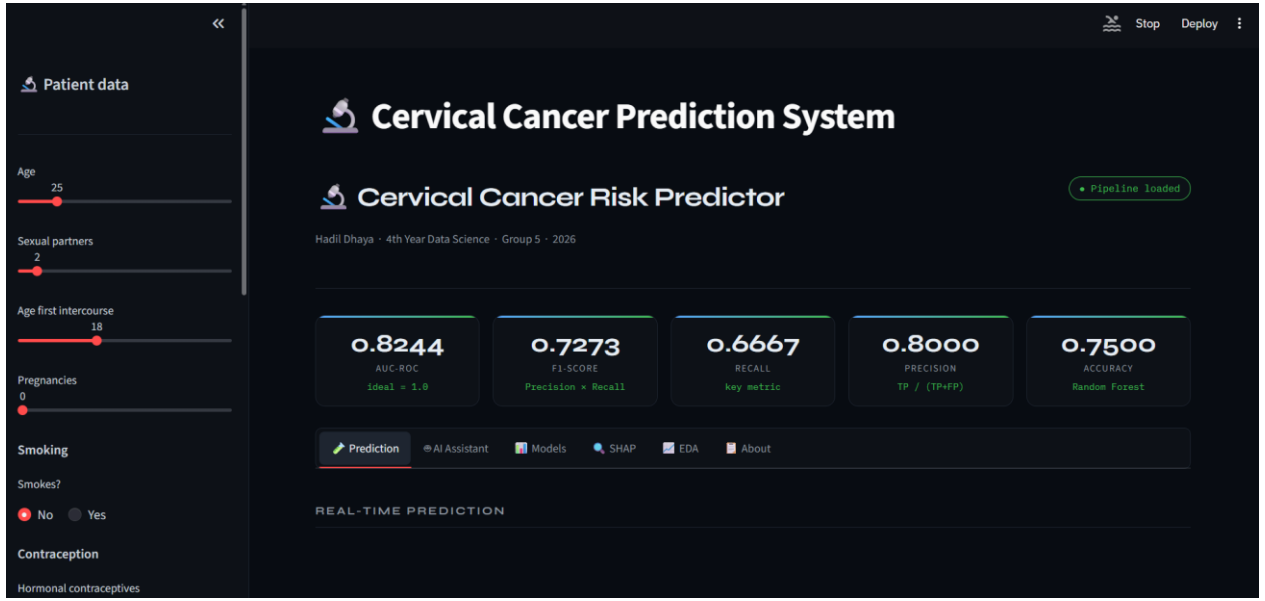
Final takeaways, deployed value, and next steps

CONCLUSION

The project achieved its main objective: building a complete ML workflow to analyze Biopsy and expose it through a usable interface.
Random Forest was the best final choice because it delivered the most stable and balanced results after cleaning and review.
Render deployment made the solution practical: users can interact with the model instead of keeping it only in notebooks.
The project contribution is not only the model — it is the full pipeline, the debugging process, the deployment, and the user experience.

FUTURE WORK

Improve data quality and collect more real samples
Tune Random Forest more deeply and test more models
Add richer explainability such as LIME or local SHAP cases
Make the chatbot more interactive and improve the Render deployment workflow



Final deployed application capture: prediction, model analysis, SHAP, EDA and chatbot tabs in one interface